

Jev AI Implementation Report

Sep 29, 2026

The one-minute summary

- **What it is:** Jev is an AI that makes quick decisions (pick one option, give a score, or say yes/no) instead of writing text. It was launched on 15 September 2026 by TypeSafe AI.
- **Why it matters:** For these small decisions it is much faster (about 0.1–0.5 seconds) and much cheaper (about \$6 a day for 100,000 decisions) than ChatGPT-style AI.
- **The catch:** It cannot write, do maths, or reason through long problems. It can still be wrong. And it is only two weeks old, so the company is still growing.
- **How to succeed:** Your own software stays in charge. Jev makes the judgment call, confident answers go ahead, and unsure ones go to a person.
- **What to watch:** accuracy, how often it is unsure, speed, and cost per correct decision.

What Jev is

Jev is an AI that **ticks boxes instead of writing essays**. ChatGPT or Claude are like an essay writer: they can write anything, but slowly. Jev is like an exam checker marking multiple-choice answers: it can't write, but it marks very fast and very cheaply.

You give Jev a situation and some questions, and you decide the possible answers in advance. It answers in one of three ways:

Question type	What it does	Everyday example
Choice	Picks one option from your list	"Which team should handle this email: billing, technical or sales?"
Score	Places something on a scale	"How angry is this customer: calm, annoyed or furious?"
Noul (yes/no)	Gives the chance something is true, from 0 to 1	"Is this email a scam?" → 0.92 means 92% likely

Every answer also comes with a **confidence score**: how sure Jev is. This is the key feature. It lets your software act when Jev is sure and ask a person when it isn't.

Quick facts

Item	Detail
Made by	TypeSafe AI , San Francisco, founded 2024 by Diogo Almeida (ex-OpenAI, worked on the research behind ChatGPT), Erik Gafni and Sasha Sheng
Launched	15 September 2026, in early access
Funding	\$40 million, led by DCVC
Current version	jev-1.13.0
Price	\$0.042 per million input tokens; answers are free
Speed	About 0.1–0.5 seconds per call
Input	Text only (no pictures, audio or video)
Best language	English; other languages work but less well
How to get it	Official site console.typesafe.ai (new signups paused since 22 Sep), Vercel AI Gateway, or OpenRouter

The name comes from economist William Stanley Jevons, who noticed that when something becomes much cheaper, people use far more of it. TypeSafe expects the same with AI decisions.

Why it exists

Companies are using big writing AIs for tiny decisions, which is like hiring a novelist to sort the mail. Sorting support tickets, spotting spam, choosing which AI should answer a question, or checking whether an AI agent is about to delete something are all short "pick one" or "yes/no" jobs. Doing them with ChatGPT-style models causes four problems. Jev is built to fix each one.

Problem with writing AIs	What Jev does instead
Slow: they write word by word, taking seconds per answer	Answers all questions at once, in about 0.1–0.5 seconds
Expensive: you pay for every word read and written	Charges only for what it reads, at a very low price; answers are free
Messy answers: sometimes the reply is in the wrong format and breaks your software	Can only return an answer from your list, so it never breaks the format

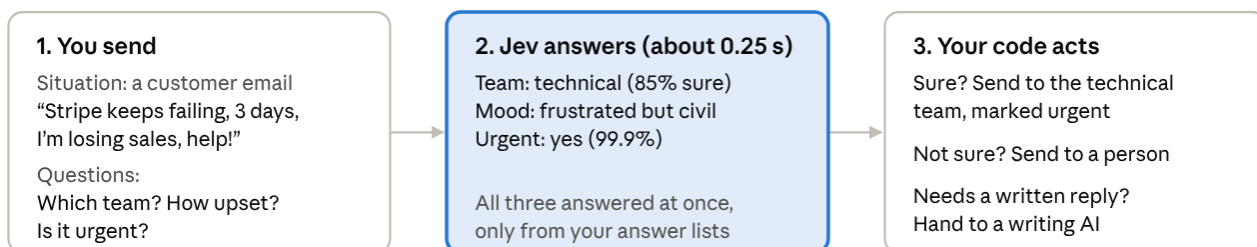
Problem with writing AIs	What Jev does instead
Overconfident: they sound sure even when wrong	Trained to give an honest confidence score, so you know when to double-check

The company's founder says the idea came from seeing that chat AIs are great at pleasing people but not reliable enough to run things on their own. Jev is his attempt at "AI that software can depend on".

How it works

Every Jev call follows the same three steps: you send a situation plus your questions, Jev answers them all at once, and your own software decides what to do.

One Jev call: a situation and questions in, ready-made answers out



one Jev call · send, answer, act

This is the example from TypeSafe's own guide. Three ideas make it work:

- **Many small questions beat one big one.** Instead of "Is this email a scam?", ask "Does it ask for a password?", "Does it promise a prize?", "Does the sender's name match the address?" Your code then combines the answers. This is more accurate and easier to fix.
- **Extra questions are nearly free.** All questions are answered together, so adding more barely slows things down.
- **Your code stays the boss.** Jev only gives judgments. Rules, maths, dates, permissions and the final action stay in normal code you control.

Where to use it and where not to

Use Jev when a decision is **quick, repeated many times, and the possible answers are known in advance**. A simple test: if a smart person could decide it in a few seconds by picking from a list, Jev can probably do it.

Good jobs for Jev

Job	What Jev decides	Proof so far
Sorting support tickets	Which team, how urgent, how upset	TypeSafe's own main example
Sorting emails	Invoice? Scam? Needs a reply?	1,000 emails, 7 checks each, in about 6 seconds for \$0.09 (MindStudio test)
Choosing which AI to use	Cheap fast AI or powerful slow AI?	Built into LangChain, a popular AI toolkit
Safety checks on AI agents	"Is this action dangerous or off-task?"	One tool checked 17,000 actions in about 0.25 s each
Filtering search results	Is this page relevant?	Best answer ranked first rose from 5% to 18% in TypeSafe's legal test
Checking AI answers	Does the source really support this claim?	Caught 4 out of 4 planted mistakes in TypeSafe's test
Tagging large piles of text	Topic, sentiment, quality	777 judgments in under 0.7 seconds (Every test)

Bad jobs for Jev (from TypeSafe's own list of weak spots)

Don't use Jev for...	Why	Use this instead
Writing or summarising	It cannot produce text	ChatGPT, Claude or similar
Maths or counting	It guesses the shape of the answer, not the exact number	Normal code
Comparing dates	It reads dates as words, not as time	Normal code
Long, multi-step reasoning	Accuracy drops with every extra step	Split into simple questions, or use a reasoning AI
Huge piles of unrelated text	Extra noise confuses it	Filter first; send only what matters
Pictures, audio, video	It only reads text	Turn them into text first
Final say on money, access or safety	It can still be wrong	Keep a person or strict rules in charge

Tradeoffs: what you gain and what you give up

Jev trades a little accuracy and a lot of flexibility for big gains in speed, cost and predictability. Whether that is worth it depends on how many decisions you make and how costly a mistake is.

The main tradeoffs

You gain	You give up	What it means for you
Speed: about 0.25 s instead of several seconds	Some accuracy: in one independent test Jev caught 6 of 7 planted mistakes; a top writing AI caught 7 of 7	Good for high-volume jobs; for rare, high-stakes decisions a slower AI may be worth it
Very low cost	No text: it can't explain its answer	Pair it with a writing AI when you need an explanation or reply
Answers always in the right format	You must design the questions carefully	More upfront thinking than just writing a prompt
Honest confidence scores	Some cases go to humans	You need a small review team or fallback for unsure cases
Managed service; no servers to run	Your data leaves your company and you depend on a 2-week-old startup	Check privacy terms; keep a backup plan
Works out of the box	Can't be trained on your own data	You steer it only through how you write questions and what you send

How Jev compares with the other options

	Jev	Big writing AI (ChatGPT, Claude)	Open-source copy (Laya)	Normal code (rules)
Speed per decision	Fast (0.1–0.5 s)	Slow (seconds)	Very fast (about 0.03 s on your own GPU)	Instant
Cost	Very low	High	Your own hardware	Almost free

	Jev	Big writing AI (ChatGPT, Claude)	Open-source copy (Laya)	Normal code (rules)
Understands messy text	Yes	Yes, best	Yes, but weaker on big option lists	No
Writes text / explains	No	Yes	No	No
Confidence score	Yes, trained for it	Unreliable	Yes	No
Data stays in-house	No	No (unless self-hosted)	Yes	Yes
Setup effort	Low	Low	Medium to high	High for complex rules
Maturity	2 weeks old	Mature	Days old	Mature

Rule of thumb: use normal code for anything exact (maths, dates, permissions), Jev for quick judgments on messy text, a writing AI for anything that needs words or deep thinking, and a person for high-stakes or unclear cases.

Metrics: what to measure

The single most important number is **confident mistakes**: how often Jev is wrong while saying it is sure. Those are the errors that slip through without a human seeing them. Track the rest to prove Jev is better, cheaper and fast enough.

Targets below are starting suggestions. Set your own by comparing with the system you use today (your **baseline**).

Quality: is it right?

Metric	In simple words	How to measure	Suggested target
Accuracy	Out of 100 decisions, how many are correct	Compare Jev's answers with a set of examples where you know the right answer	Equal to or better than today's system

Metric	In simple words	How to measure	Suggested target
Confident mistakes	Wrong answers where Jev said it was sure	Count wrong answers above your confidence threshold	As low as possible; under 1–2% for anything customer-facing
False alarms	Jev says "yes" (e.g. scam) when it's actually "no"	Count on your test set	Depends on cost of a false alarm
Misses	Jev says "no" when it's actually "yes"	Count on your test set	Lower for safety checks than for convenience features
Calibration	When Jev says 90% sure, is it right about 90% of the time?	Group answers by confidence and check accuracy in each group	Accuracy rises steadily with confidence
Consistency	Same kind of input gives the same answer	Re-run the same examples	Near-identical results

Operations: does it run well?

Metric	In simple words	Suggested target
Automation rate	Share of decisions Jev handles alone, with no human	Grows over time; 70–90% is a common aim for triage jobs
Human review rate	Share sent to a person because Jev was unsure	Small enough for your team to handle
Speed (typical, p50)	How long half of the calls take	Under 0.3–0.5 s
Speed (slow cases, p95)	How long the slowest 5% take	Within your app's time limit; measure from India, not just the US
Error and "too busy" rate	Calls that fail or get rate-limited	Under 1%, with automatic retries

Business: is it worth it?

Metric	In simple words	How to measure
Cost per correct decision	Everything you spend divided by decisions that turned out right	(Jev bill + retries + human review time + backup AI costs) ÷ correct decisions
Savings vs. today	Money and time saved compared with the old way	Same calculation for your current system
Outcome impact	Did the real goal improve?	E.g. ticket reply time, spam reaching inboxes, agent accidents avoided

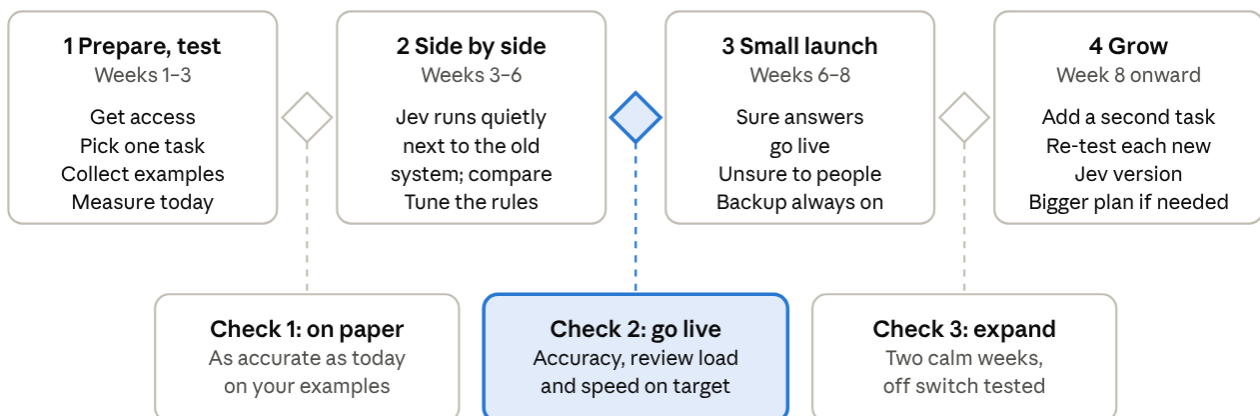
Why "cost per correct decision" and not "cost per call": a very cheap call that sends 30% of cases to humans can cost more overall than a pricier one that sends 5%.

How to implement: step by step

A first task can be running for real in about eight weeks, with three checkpoints where you decide whether to continue.

Jev goes live only after it passes the side-by-side test

Rough timing for one task; each checkpoint is a go / no-go decision



implementation timeline · 4 phases, 3 checkpoints

Timings are rough and assume you already have past examples with known answers. If a checkpoint fails, stay in that phase and improve the questions or rules — or stop and keep your current system.

The steps in detail

1. **Get access safely.** Sign up at console.typesafe.ai, or use Vercel AI Gateway or OpenRouter while signups are paused. Keep the API key on your server, never in an app or browser.
2. **Pick one task.** Choose something frequent, low-risk and easy to undo, like sorting support tickets.
3. **Write the possible answers.** List every option, and always add an "unclear / other" option so Jev has an honest way out.
4. **Collect 200–500 real examples** where you already know the right answer. Include tricky, rare and non-English cases.
5. **Measure today's system** on those same examples: accuracy, speed and cost. This is your baseline.
6. **Write short, simple questions.** One idea per question. Send only the information each question needs. Do maths and dates in normal code.
7. **Try it in the Playground**, then connect with the official Python or JavaScript kit (or LangChain if you already use it).
8. **Set the confidence rules.** For example: above 0.8 act automatically, below that send to a person. Use stricter rules for risky actions. Tune these using your examples.
9. **Run side by side** for 2–4 weeks. Jev makes decisions but doesn't act; compare with the old system and study every disagreement.
10. **Go live in small steps**, with the backup always ready and a one-click "off switch".
11. **Keep watching the metrics** and re-test before moving to any new Jev version.

Costs

The Jev bill itself is tiny; the real costs are your team's time to set it up, test it, and review the unsure cases.

How pricing works. Jev charges \$0.042 for every million "tokens" it reads (a token is roughly three-quarters of a word). The answers it gives back are free. New accounts got \$5 of credit, which covers about 119 million tokens.

Worked examples (assuming each decision reads about 1,500 tokens, roughly a long email plus your questions)

Size	Decisions per day	Jev cost per day	Jev cost per month	Same reading on a writing AI (input only)
Small pilot	10,000	\$0.63	about \$19	\$3–\$150 per day
Team scale	100,000	\$6.30	about \$189	\$30–\$1,500 per day

Size	Decisions per day	Jev cost per day	Jev cost per month	Same reading on a writing AI (input only)
Large company (2,000 tokens each)	1,000,000	\$84	about \$2,520	\$400–\$20,000 per day

The writing-AI column uses the \$0.20–\$10 per million range TypeSafe quotes for other models, and leaves out their output charges, so the real gap is bigger. Your token counts will differ; measure them in the pilot.

Other costs to budget for

- **Engineering time** to design questions, connect the API and build the confidence rules
- **Labelling time** to build a test set of a few hundred examples with known right answers
- **Human review** of the cases Jev isn't sure about
- **Backup AI** calls for cases that need writing or when Jev is unavailable
- **Enterprise plan** if you need higher limits or "zero data retention" (price on request)

Limits to know. Today's limits are 1,200 calls per minute and 250,000 tokens per second, and TypeSafe says these can change without notice while demand is high. Team scale fits easily; very large use needs batching or an enterprise plan.

Price warning. TypeSafe admits it can't yet prove the price isn't subsidised, though it expects prices to fall. Don't build a business case that only works at today's price.

Risks and how to handle them

The biggest risks are trusting Jev too much and depending on a very young company. Each one has a simple fix.

Risk	What could happen	How to handle it
Wrong answer that looks right	Jev picks a valid option that is simply wrong	Use confidence scores; add an "unclear" option; send unsure cases to a person; test before going live
The company is overloaded	Signups are paused and limits change without warning	Get an account early; keep a backup (another AI or simple rules); consider a second route like Vercel
Silent changes	TypeSafe releases a new version and answers shift	Lock to version <code>jev-1.13.0</code> ; re-test before switching

Risk	What could happen	How to handle it
Tricked by sneaky text	A message written to fool Jev changes its answer	Treat customer text as untrusted; add a "is this trying to manipulate?" check; never let Jev alone approve money or access
Claims don't match your data	The "193× faster, 444× cheaper" figures come from TypeSafe's own tests	Decide based on your own test results only
Weaker in non-English text	Hindi or other languages may be less accurate	Test each language separately
Fake Jev websites	Copycat sites (e.g. jev-ai.pro, jevtypesafeai.com) charge up to 11.5× more and see your data	Use only typesafe.ai, console.typesafe.ai and api.typesafe.ai
Price rise or company failure	Costs change or the service shuts down	Wrap Jev behind your own code so you can swap it; keep an open-source option (Laya) tested
Slower from India	Servers are in the US, adding network delay	Measure speed from your location; avoid it in the tightest real-time paths

Privacy and data. TypeSafe says it does not train on customer data. Its data agreement promises to report security breaches within 72 hours and covers EU and UK rules. Deleting data straight away ("zero data retention") is only for enterprise customers. If you send personal data about people in India, check your duties under the Digital Personal Data Protection Act, 2023 with a lawyer, and send as little personal data as possible. I could not confirm whether TypeSafe has security certificates such as SOC 2, so ask for them.

Sources

Speed and cost multipliers are TypeSafe's own figures unless marked as an independent test.

- [TypeSafe AI](#) and its [launch post](#)
- TypeSafe docs: [introduction](#), [quick start](#), [models and pricing](#), [known weak spots](#), [confidence](#)
- [TypeSafe data processing agreement](#)

- [Wikipedia: Jev \(AI model\)](#)
- [Firecrawl: What is Jev?](#) (independent Every test, pi-warden, access routes)
- [MindStudio: 12 Jev use cases tested](#)
- [Wavect: Jev AI review](#)
- [TechStock²: cost claims are self-tested](#)
- [LangChain: Building a harness with Jev](#)
- [Eye Security: fake Jev websites](#)
- [Wavect: Laya vs Jev](#)